

When a Line Is Not Enough

Unit 5: Fixing a Model, and Letting a Tree Show You Where to Look

Stat 220 | Statistics for Data Science

Where we left off

- Units 3 and 4 fitted straight lines, read the coefficients, and chose which predictors to keep.
- All of that assumed the straight line was roughly the right shape, and that the spread around it was roughly constant.
- This unit is what you do when it is not: how to tell, what to change, and what to reach for when a line will not do the job at all.

What this unit gives you

Things to know

- what each broken assumption looks like in a residual plot
- when a log fixes spread that grows, and how to read the result
- how to put curves and interactions into a linear model
- how a decision tree is built, one split at a time
- what a tree is good at, and where it falls down
- how to use a tree to find terms worth adding to a regression

Mistakes to stop making

- reading coefficients off a model whose residuals are patterned
- taking logs and then reporting the coefficient in the original units
- adding squared terms everywhere instead of looking first
- reading a tree's splits as effects
- expecting a tree to predict outside the range of the data
- choosing between a tree and a regression on their fit to the same rows

The data for this unit

diamonds.csv: 5,000 stones from a jeweller's price list, real data.

Column	What it is
price	what it sold for, \$351 to \$18,787, the outcome
carat	weight, 0.23 to 4.13
cut	Fair, Good, Very Good, Premium, Ideal
color	D (best) through J
clarity	IF (flawless) through I1
depth, table, x, y, z	proportions and measurements

Price against weight breaks the straight-line model in two ways at once, and fixing it turns up something true about how diamonds are sold.

What the residuals are telling you

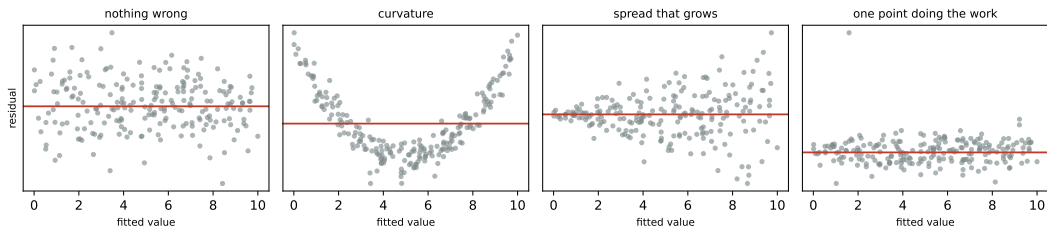
Plot the residuals against the fitted values. Four things turn up, and each one has a standard response.

What you see	What it means
a shapeless cloud	the shape you fitted is about right
a curve or a U	the relationship bends and the line does not
a fan or a wedge	the spread grows with the size of the prediction
one point far away	a single case is pulling the fit toward itself

Principle: look before you read

Plot the residuals before you read a single coefficient. The coefficient table cannot tell you whether the shape you fitted was the right one.

The four pictures



Only the first is a model you can read coefficients from as it stands. The next slide gives the response to each of the other three.

What to do about each one

Problem	Usual response
the relationship bends	add a squared term, or transform the predictor
spread grows with the fit	model $\log(y)$ instead of y
one case dominates	find out what it is; report the fit with and without it
cases are not independent	a different family of models, named on the next slide
the outcome is not a number	also a different family, also next slide

Trap: the quiet failure

None of this shows up in the output. A model with patterned residuals still prints coefficients, p-values and an R^2 to three decimals.

The same four responses, in code

What you saw	What to type
always, first	<pre>plt.scatter(fit.fittedvalues, fit.resid) plt.axhline(0)</pre>
a curve	<pre>smf.ols("y ~ x + I(x**2)", data)</pre>
a fan	<pre>smf.ols("np.log(y) ~ x", data)</pre>
a curve <i>and</i> a fan	<pre>smf.ols("np.log(y) ~ np.log(x)", data)</pre>
one point far out	<pre>df.loc[fit.resid.abs().idxmax()] then go and find out what it is</pre>

The diamonds show a curve and a fan together, which is why the fix there is the fourth line.

Three things this course points at but does not teach

- **Time series.** Measurements on the same thing over time are not independent: today looks like yesterday. Unit 1 showed what that does to a p-value. Fixing it properly needs models built for it.
- **Spatial statistics.** The same problem in two dimensions: nearby places resemble each other. Mowing times on one street are not independent draws.
- **Generalized linear models.** When the outcome is a yes/no or a count, the model gets a different link and a different spread. Logistic regression, which Unit 10 covers, is one of these.

In practice: what to do when you meet one

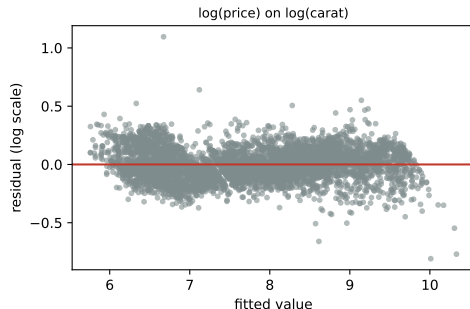
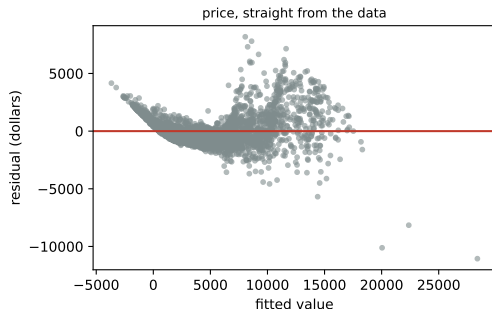
Name it, say so in the write-up, and go find the right tool or the right person. Recognizing that your rows are not independent is the part you are responsible for here.

Why spread so often grows with the prediction

- A second carat does not add a fixed number of dollars. Price scales with weight, and the premium for clarity is a percentage of a bigger number.
- When effects multiply rather than add, the spread around a large prediction is larger than the spread around a small one. That is the fan.
- Taking logs turns multiplication into addition, which is exactly the shape a linear model is built for.

$$y = a \cdot b^x \cdot \varepsilon \quad \implies \quad \log y = \log a + x \log b + \log \varepsilon$$

The same jobs, before and after



Left: price on the raw scale. The spread at an expensive prediction is about twice the spread at a cheap one, and the residuals arc as well. Right: log(price) on log(carat). The arc is gone, the spread is even from end to end, and R^2 rises from 0.916 to 0.983.

Reading a coefficient after a log

- On a logged outcome, a coefficient b is a *percentage*: roughly $100(e^b - 1)\%$ for each one-unit change in the predictor.
- If you log the predictor too, the coefficient is a percentage for a percentage. Here the coefficient on $\log(\text{carat})$ is **1.88**: a stone 10% heavier costs about 19% more.
- That number has a name outside this course. Economists call it an elasticity, and it is the same idea.

Trap: reporting in the wrong units

“Carat adds 1.88” is not a sentence about dollars, and a buyer will read it as one. Convert before you report.

Which log to take

Take	When	A coefficient of 0.2 means
$\log(y)$	the spread fans out, and x is a count or a flag	about 22% more y per unit of x
$\log(x)$	y rises quickly and then flattens	about 0.14 more y each time x doubles
both	the cloud bends <i>and</i> fans, the usual case for prices and sizes	0.2% more y per 1% more x

In practice: how to decide

Fit it both ways and compare the residual plots. The version that leaves no pattern wins, and it is usually the one the subject matter already suggested.

Two things to know before you take a log

- **Zeros and negatives have no log.** Counts that include zero need a different answer, not $\log(y + 1)$ chosen because it ran without an error.
- **Exponentiating a fitted value gives you back a median, not a mean.** For most operational questions that is fine, and it is worth knowing which one you handed over.
- The model is now about $\log(\text{price})$. Every interval and every prediction comes back on that scale until you convert it.

Your turn #1

Your turn: which fix fits which picture?

Match each residual plot to its response:

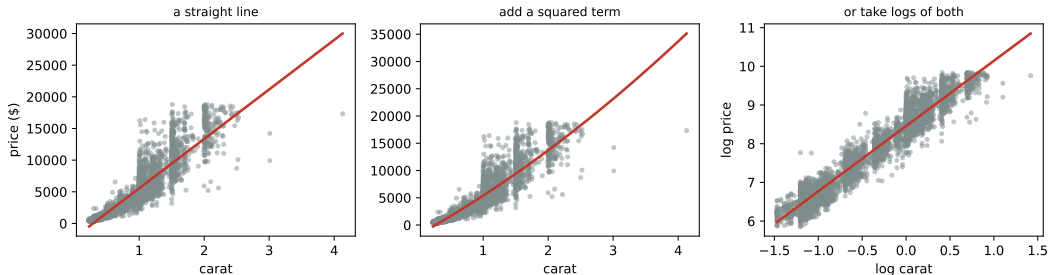
- 1 The residuals fan out as the fitted value grows.
- 2 The residuals form a clear arc, negative at both ends.
- 3 One residual sits five times further from zero than any other.

Decide with a neighbour, and say what you would check before doing anything to number 3.

Your turn #1: answer

- **(1) Fan.** Model the log of the outcome, then report in percentages.
- **(2) Arc.** The shape is wrong, not the spread. Add a squared term or transform the predictor.
- **(3) One point.** Find out what it is first. A data entry error gets fixed, a real unusual job gets reported both ways, and neither gets deleted because it was inconvenient.

Bending a line on purpose



The same 5,000 stones three ways. A straight line misses badly at both ends: it says a quarter carat is worth negative money. A squared term follows the curve. Taking logs of both sides straightens the relationship instead of bending the model, which is the better move when the cause of the curve is multiplicative.

The two tools you already have

- **Polynomial terms.** Add x^2 , and the fitted curve can bend once. Add x^3 and it can bend twice. Each is an ordinary column, and the model is still linear regression.
- **Interactions.** Add x_1x_2 , and the effect of one predictor is allowed to depend on the other. Steep lots may cost more time only when they are also large.
- Both are choices *you* make about shape, before fitting.

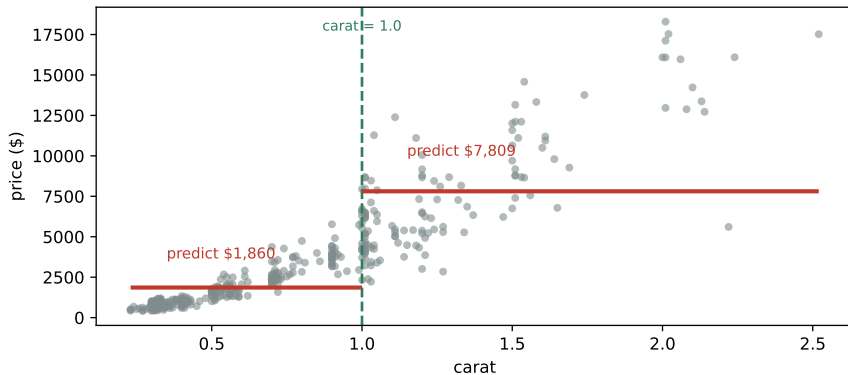
Principle: the catch

A squared term only helps if you suspected a curve in that variable. An interaction only helps if you suspected those two variables interact. With a dozen predictors there are far more possibilities than you can check by hand.

A different idea entirely

- Stop writing an equation. Instead, ask a sequence of yes/no questions and read the answer off the end.
- *Is the stone heavier than one carat? If yes, is its clarity SI2 or worse? If yes, predict \$4,574.*
- That is a **decision tree**. The questions are chosen by the data, not by you, which is the point.

One split



Cut the stones into two groups at one carat. Predict the average of each group. The prediction is now a step function with two steps, and already it beats guessing the overall average.

How the cut is chosen

For a candidate cut, score it by how much of the spread is left inside the two groups:

$$\text{score} = \sum_{\text{left}} (y_i - \bar{y}_{\text{left}})^2 + \sum_{\text{right}} (y_i - \bar{y}_{\text{right}})^2$$

- Try every cut point of every predictor. Keep the one with the smallest score.
- Then do the same thing again inside each group, and again inside those.
- There is nothing subtle about it. The machine tries every option and keeps the best one.

Principle: greedy, one step at a time

Each split is chosen as if it were the last one. The tree never goes back and reconsiders an earlier cut, which is why a slightly different sample can produce a rather different tree.

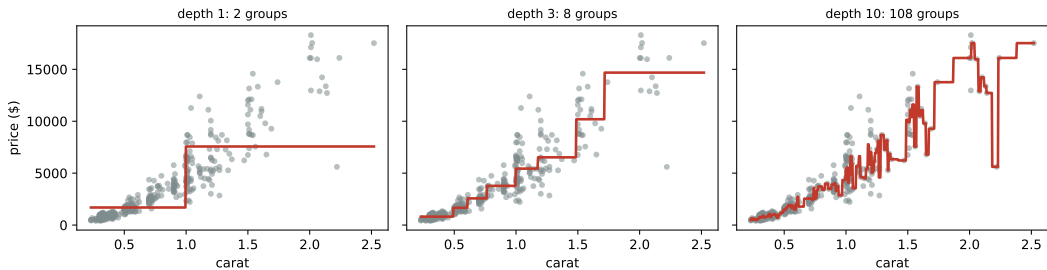
The same arithmetic, in numbers

Three candidate cuts on carat, scored on the same stones:

Cut at	left average	right average	spread left over
0.5 carat	\$1,100	\$5,900	44.1 billion
1.0 carat	\$2,000	\$8,600	30.6 billion
1.5 carat	\$2,800	\$11,400	36.9 billion

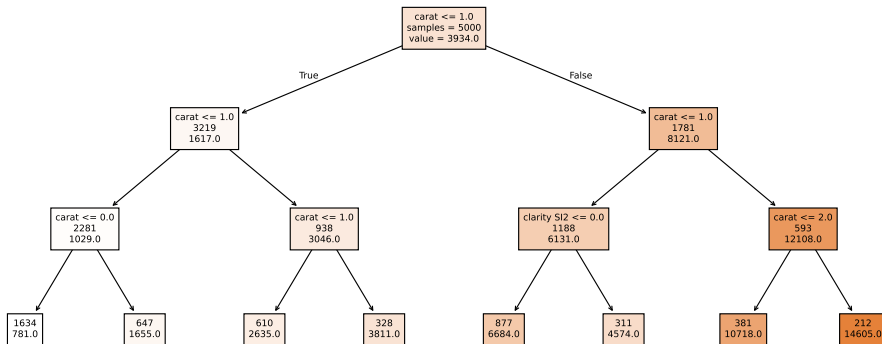
The middle cut wins, so one carat is the first question the tree asks. The machine does this for every cut point of every variable before choosing.

Keep splitting



One split gives two steps. Three levels give eight price bands and a recognisable staircase. Ten levels give a step for almost every handful of stones. At that point the tree is describing these 400 stones rather than the market they came from.

Reading a fitted tree



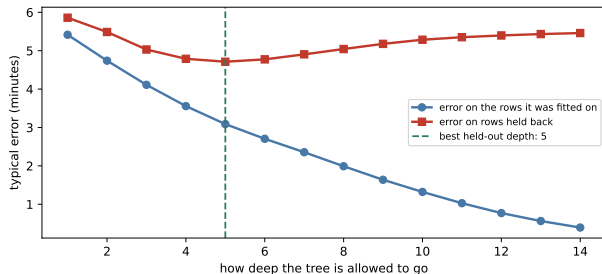
The same tree, out loud

- The first question is carat, at exactly **1.00**. Weight matters more than everything else combined, which matches the regression.
- Almost every other split is also carat: 0.47, 0.61, 0.81, 1.38. The tree is carving the weight axis into price bands.
- Inside the heavy branch it finally asks about **clarity**. Among stones over a carat, the SI2 ones sell for about \$2,000 less.
- Each box at the bottom holds a group of stones and predicts their average.

Principle: what a tree is

A set of rules that partition the data, and one number per region. No coefficients, no equation, and nothing that can be read as the effect of a variable.

How deep to go



This one is a different dataset: 700 lawn mowing jobs, where the signal is weaker.

- Error on the fitted rows falls with every level and keeps falling.
- Held-out error bottoms out around depth 5 and then climbs.
- On the diamonds this turn barely happens: carat predicts price so well that extra depth costs little. The weaker the signal, the more the depth matters.

What trees are good at, and bad at

Good at	Bad at
thresholds, where something changes at a value	smooth trends, which it approximates with stairs
interactions, free of charge, because every split happens inside an earlier split	giving you an effect size with an interval
predictors on any scale, with no transformation needed	stability: a few different rows can change the top split
mixed types of predictor in one model	anything outside the range of the data

The extrapolation problem, in one line

- A tree predicts the average of the training rows in a region. For a stone heavier than any it has seen, it returns the average of the heaviest group it knows.
- A line at least continues in a direction. A tree flattens out completely.
- Neither is trustworthy out there, but they fail differently, and the tree fails silently.

Forests and boosting, in one slide

- A single tree is unstable. **Random forests** fit many trees to resampled data and average them. **Boosting** fits trees in sequence, each correcting the last.
- Both usually predict better than one tree, and both give up the one thing a single tree had: you could read it.
- We are not going further than naming them. Unit 2 put them on the menu, and this is where they come from.

Your turn #2

Your turn: what would the tree do?

A tree is fitted to diamond prices. Its first split is *carat above 1.00*, and inside the heavy branch it splits on *clarity SI2 or worse*.

A 2.5 carat stone of SI2 clarity comes in, heavier than anything in the data. What does the tree predict, and what would the log-log regression do differently?

Your turn #2: answer

- The tree sends it right, then right again, and predicts the average of the heaviest SI2 stones it has seen. For a 2.5 carat stone that average is too low, and it would be the same answer for a 4 carat stone.
- The regression keeps going: 1.88% more price for every 1% more weight, wherever you point it. That is also an extrapolation, and also not trustworthy.
- The difference is the shape of the failure. The regression keeps rising; the tree returns a flat number and gives no sign that it has run out of data.

Using a tree to improve a regression

- A tree will not give you an effect with an interval, so it cannot be the thing you report when the question is how much something matters.
- But it searches shapes you would never check by hand, and its splits are hypotheses written in a readable form.
- So: fit the tree, read the structure, and put what it found into the regression.

Principle: scout and report

Use the tree to find where to look. Use the regression to say how much, and how sure.

Turning splits into terms

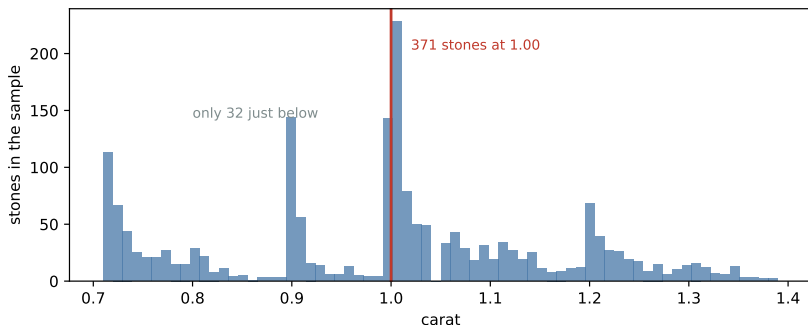
What you see in the tree, and what you add to the regression because of it:

In the tree	In the regression
one split on x at a round or meaningful value	an indicator: $I(x \geq c)$
several splits on the same x at different values	a curve: $I(x^2)$, or $\text{np.log}(x)$
a split on x_2 inside one branch of x_1 , and not the other	an interaction: $x_1 * x_2$
a split on a category inside one branch only	that category crossed with the branching variable
a variable the tree never splits on	a candidate for dropping, after you check it is not just correlated with one it did use

What this tree was telling us

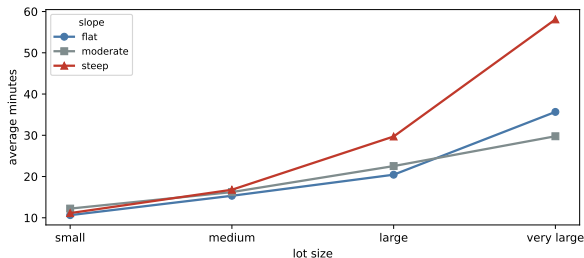
- **The very first split is at 1.00 carat**, not at 0.9 or 1.1. A model that is smooth in weight cannot produce a jump at a round number, so either the tree is chasing noise or something real happens there.
- **Splits on carat at several other places.** The relationship with weight is not a straight line, which the log already handled.
- **Clarity appears only among the heavy stones.** A hint that clarity matters more as weight goes up.

The market leaves a fingerprint



There is a pile of stones at exactly 1.00 carat and almost none just below. Cutters grind away weight to reach the round number, because buyers shop for “a one carat diamond”. The split the tree chose is a fact about the trade.

The same move, on an interaction



A second dataset, the lawn jobs, where the tree split on steep slope only inside the large-lot branch.

- On small and medium lots the three slopes cost about the same.
- On large lots, steep pulls away.
- A split that appears in one branch and not another is the tree's way of writing an interaction.

Put it in the model

Add one indicator, *is the stone at least one carat*, to the log-log model:

Model	R^2	AIC	cross-validated error
price on carat, no logs	0.916		0.337
log price on log carat	0.983	−5842	0.135
plus the one carat jump	0.984	−6126	0.131
the tree on its own (depth 5)			0.228

- The jump is **12%** in price, at the same weight, clarity, colour and cut: $p \approx 10^{-64}$, and AIC improves by 284.
- The regression that took the hint predicts better than the tree that gave it, and it hands you a number a buyer can use.

Saying it out loud

The same finding, written three ways. Only the last one is usable.

Too raw	"The over1 coefficient is 0.115 with $p < 0.001$."
Too loose	"One carat diamonds are way more expensive."
Right	"Crossing one carat adds about 12% to the price, on top of what the extra weight already costs. A 0.98 and a 1.01 carat stone are nearly the same rock, and the heavier one runs about 19% more: 6% for the weight, 12% for the round number. If you are buying, shop just under the line."

In practice: the pattern

The number, the units, what is held fixed, and what somebody should do about it.
Four parts, two or three sentences.

Why end here

- The tree is a fast, honest search for structure you did not think to look for.
- The regression turns what it finds into a number with an interval, in units somebody can act on.
- Used together they cover each other's weakness, and you can defend every term in the final model by saying where it came from.

Trap: the step you cannot skip

A term suggested by a tree was still suggested by looking at the data. Check it on held-out rows, and say in the write-up that this is where it came from.

LLM check: mistake or not? (1 of 2)

LLM check: we asked about a fan in the residuals

We told it: “My residuals fan out as the fitted values grow. What should I do?”

It answered: “Remove the outliers at the high end and refit. That will stabilise the variance and improve your R^2 .”

Mistake, or no mistake? Decide before we click.

LLM check: mistake or not? (1 of 2)

LLM check: we asked about a fan in the residuals

We told it: “My residuals fan out as the fitted values grow. What should I do?”

It answered: “Remove the outliers at the high end and refit. That will stabilise the variance and improve your R^2 .”

Mistake, or no mistake? Decide before we click.

Trap: verdict: mistake

A fan is not an outlier problem. The large cases are not errors, they are simply more variable, and deleting them changes the question the model answers. Model the log instead.

LLM check: mistake or not? (2 of 2)

LLM check: we asked it to read a tree

We told it: “My tree splits on gated right after lot size. What does that tell me?”

It answered: “Gating is the second most important driver of mowing time. The split suggests it matters within a size range, so it may be worth testing a gate term, or a gate by size interaction, in a regression where you can put a number on it.”

Mistake, or no mistake?

LLM check: mistake or not? (2 of 2)

LLM check: we asked it to read a tree

We told it: “My tree splits on gated right after lot size. What does that tell me?”

It answered: “Gating is the second most important driver of mowing time. The split suggests it matters within a size range, so it may be worth testing a gate term, or a gate by size interaction, in a regression where you can put a number on it.”

Mistake, or no mistake?

Principle: verdict: no mistake

The second sentence is exactly the move this unit ends on. The phrase “second most important” is loose, but it stops short of calling the split an effect.

What to do when a line is not enough

- 1 **Plot the residuals** against the fitted values, every time.
- 2 **Name what you see:** cloud, curve, fan, or one point.
- 3 **Curve:** add a squared term, or transform the predictor.
- 4 **Fan:** model the log of the outcome, and report in percentages.
- 5 **One point:** find out what it is before touching it.
- 6 **Not independent, or not a number:** name the right family and get help.
- 7 **Out of ideas:** fit a shallow tree and read its splits for terms to try.
- 8 **Check any term you found this way** on data the search did not touch.

The traps, all in one place

Trap: the ones that bite most often

- Reading coefficients from a model whose residuals are patterned.
- Reporting a coefficient from a logged model in the original units.
- Using $\log(y + 1)$ on zeros because it ran without complaining.
- Deleting an inconvenient point and not saying so.
- Calling a tree's split an effect, or its variable ranking an effect size.
- Asking a tree to predict outside the range of its training data.
- Comparing a tree and a regression on the rows both were fitted on.
- Adding a tree-suggested term and reporting its p-value as if you had planned it.

What you should be able to do with software

You should be able to	What you would reach for
Plot residuals against fitted values	<code>plt.scatter(fit.fittedvalues, fit.resid)</code>
Model a logged outcome	<code>smf.ols("np.log(y) ~ x", data)</code>
Add a curve	<code>smf.ols("y ~ x + I(x**2)", data)</code>
Add an interaction	<code>smf.ols("y ~ x * group", data)</code>
Fit a shallow tree	<code>DecisionTreeRegressor(max_depth=3).fit(X, y)</code>
Read the tree	<code>export_text(tree, feature_names=...)</code>
Choose the depth	<code>cross_val_score</code> over a few depths

Where we go next

- That closes the modelling half of the course. You can fit a model, read it, check it, repair it, choose between candidates, and predict with honest error bars.
- **Unit 6** starts the machinery underneath all of it: probability, and what chance alone produces.
- Everything from here gives a reason for the tools you have been using, and then puts them to work on outcomes that are not continuous numbers.